

[← Go Back](#)

Hardware

Nano 33 IoT



Tutorials

Controlling a LED Through Bluetooth® with Nano 33 IoT

Connecting Two Nano 33 IoT Boards Through I2C

Sending an Email Through IFTTT with Nano 33 IoT

Accessing Accelerometer Data on Nano 33 IoT

Accessing Gyroscope Data on Nano 33 IoT

Nano 33 IoT Alarm with the Arduino Cloud

Connecting Two Nano 33 IoT Boards Through UART

Connecting the Nano 33 IoT to a Wi-Fi Network

[Home](#) / [Hardware](#) / [Nano 33 IoT](#) /

Controlling a LED Through Bluetooth® with Nano 33 IoT

Controlling a LED Through Bluetooth® with Nano 33 IoT

Learn how to control the built-in LED on the Nano 33 IoT board over Bluetooth®, using an app on your phone.

Author · [Fabricio Troya](#) · Last revision · 07/17/2024

In this tutorial we will use an Arduino Nano 33 IoT to turn on the built-in LED over Bluetooth®, made possible by the NINA module embedded on the board.

Goals

The goals of this project are:

Learn what Bluetooth® Low Energy and Bluetooth® are.

Use the Arduino BLE library.

Learn how to create a new service.

Learn how to control a LED from an external device (smartphone).

Hardware & Software Needed

This project uses no external sensors or

ON THIS PAGE

Goals

Hardware & Software Needed

Bluetooth® Low Energy and Bluetooth®

Service & Characteristics

Unique Universal Identifier (UUID)

Creating the Program

Testing It Out +

Conclusion

[Help](#)

In this tutorial we will use the [Arduino Cloud Editor](#) to program the board.

Bluetooth® Low Energy and Bluetooth®

Bluetooth® Low Energy, referred to as Bluetooth® Low Energy, separates itself from what is now known as “Bluetooth® Classic” by being optimized to use low power with low data rates. There are two different types of Bluetooth® devices: central or peripheral. A central Bluetooth® device is designed to read data from peripheral devices, while the peripheral devices are designed to do the opposite. Peripheral devices continuously post data for other devices to read, and it is precisely what we will be focusing on this tutorial.

Service & Characteristics

A service can be made up of different data measurements. For example, if we have a device that measures wind speed, temperature and humidity, we can set up a service that is called “Weather Data”. Let’s say the device also records battery levels and energy consumption, we can set up a service that is called “Energy information”. These

Characteristics are components of the service we mentioned above. For example, the temperature or battery level are both characteristics, which record data and update continuously.

Unique Universal Identifier (UUID)

When we read data from a service, it is important to know what type of data we are reading. For this, we use UUIDs, who basically give a name to the characteristics. For example, if we are recording temperature, we want to label that characteristic as temperature, and to do that, we have to find the UUID, which in this case is "2A6E". When we are connecting to the device, this service will then appear as "temperature". This is very useful when tracking multiple values.

If you want to read more about UUIDs, services, and characteristics, check the links below:

[GATT services.](#)

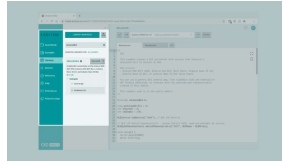
[GATT characteristics.](#)

Creating the Program

1. Setting up

Let's start by opening the [Arduino Cloud Editor](#) click on the

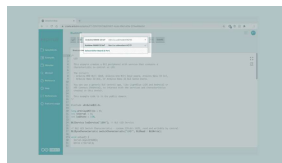
ArduinoBLE library. Then in **Examples > Peripheral**, open the **LED** sketch and once it opens, you could rename it as desired.



Finding the library in the Cloud Editor.

2. Connecting the board

Now, connect the Arduino Nano 33 IoT to the computer and make sure that the Cloud Editor recognizes it, if so, the board and port should appear as shown in the image below. If they don't appear, follow the [instructions](#) to install the plugin that will allow the Editor to recognize your board.



Selecting the board.

3. Turning ON the LED

Now we will need to modify the code on the example, in order to control LED, based on the information we sent through the smartphone.

After including the `ArduinoBLE.h` library, in the `ledService()` function change the argument to `"180A"` which is translated to **"Device**

function change the argument to `"2A57"` which is translated to **"Digital Output"**.

```
1  BLEService ledService("
2
3  // BLE LED Switch Chara
4  BLEByteCharacteristic s
```

In the `setup()` after initializing the serial communication we set the LED pin as `OUTPUT`.

```
1  Serial.begin(9600);
2  while (!Serial);
3
4  // set built in LED p
5  pinMode(LED_BUILTIN,
```

Then, inside the `setup()` function, we need to go to the `BLE.setLocalName()` and change the name to `"Nano 33 IoT"` in order to identify the board we are using.

```
1  // set advertised local
2  BLE.setLocalName("Nan
3  BLE.setAdvertisedServ
```

Now, in the `loop()`, we replace the second `if()` function inside the `while` loop with a `switch case` function, where each case will control the LED in a different manner:

Case 01 will turn the LED
ON

Case 02 will blink the LED
each 500 ms

Case 03 will blink the LED
each second

```
1 // while the central
2 while (central.conne
3 // if the remote de
4 // use the value to
5 if (switchCharacter
6     switch (switchCha
7     case 01:
8         Serial.printl
9         digitalWrite(
10        break;
11    case 02:
12        Serial.pri
13        digitalWrite
14        delay(500);
15        digitalWrite
16        delay(500);
17        digitalWrite
18        delay(500);
19        digitalWrite
20    break;
21    case 03:
22        Serial.printl
23        digitalWrite(
24        delay(1000)
25        digitalWrite
26        delay(1000)
27        digitalWrite
28        delay(1000)
```

Lastly, after the last

`Serial.println()` function we
use the `digitalWrite()` to turn
off the LED once the phone is
disconnected from the board.

```
1 // when the central dis
2   Serial.print(F("Dis
3   Serial.println(cent
4   digitalWrite(LED_BU
5   }
6 }
```

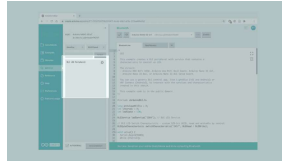
4. Complete code

If you choose to skip the code building section, the complete code can be found below:

```
1  #include <ArduinoBLE.h>
2
3  long previousMillis = 0;
4  int interval = 1000;
5  int ledState = LOW;
6
7  BLEService ledService("18155752-4F31-482D-BF68-6F4665F7943F");
8
9  // BLE LED Switch Characteristic
10 BLEByteCharacteristic ledCharacteristic;
11
12 void setup() {
13   Serial.begin(9600);
14   while (!Serial);
15
16   // set built in LED pin
17   pinMode(LED_BUILTIN, OUTPUT);
18
19   // begin initialization
20   if (!BLE.begin()) {
21     Serial.println("BLE not found");
22   }
23   while (1);
24 }
25
26 // set advertised local name
27 BLE.setLocalName("ArduinoBLE");
28 BLE.setAdvertisedService(ledService);
```

Testing It Out

to the board. When it has successfully uploaded, open the Serial Monitor. In the Serial Monitor, the text "**BLE LED Peripheral**" will appear as seen in the image below.



Serial Monitor output.

We can now discover our Nano 33 IoT board in the list of available Bluetooth® devices. To access the service and characteristic we recommend using the **LightBlue** application. Follow [this link for iPhones](#) or [this link for Android phones](#).

Once we have the application open, follow the image below for instructions:



Accessing through a Bluetooth® phone app.

To control the LED, we simply need to write 1,2 or 3 in the "WRITTEN VALUES" field to turn on the LED, make the LED blink fast or slow and any other value to turn them off. This is within the "**Digital Output**" characteristic, which is located under "**Device Information**".

Sometimes errors occur, if the code is not working there are some common issues we can troubleshoot:

Missing a bracket or a semicolon.

Arduino board connected to the wrong port.

We haven't opened the Serial Monitor to initialize the program.

The device you are using to connect has its Bluetooth® turned off.

Conclusion

In this tutorial we have created a basic Bluetooth® peripheral device. We learned how to create services and characteristics, and how to use UUIDs from the official Bluetooth® documentation. Lastly, we control the LED based on the values sent from the smartphone.

Now that you have learned a little bit how to use the ArduinoBLE library, you can try out some of our other tutorials for the Nano 33 IoT. You can also check out the [ArduinoBLE](#) library for more examples and inspiration for creating Bluetooth® projects!

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated	Help Center Ask the Arduino Forum Discover	The Arduino documentation is licensed under the Creative Commons

GitHub
repository.
If you see
anything
wrong, you
can edit
this page
[here](#).

Was this article helpful?

